1 Roteiro Prático 05 (Parte 1) — Assincronismo, Fetch API & Microbackend Flask

Objetivos de Aprendizagem — Roteiro 05 (Parte 1: Fundamentos & Mini-Exemplos)

Objetivos de Aprendizagem (Módulo 3 — Assincronismo & Integração de Serviços Web)
Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Compreender a transição do armazenamento isolado no navegador (`localStorage`) para a **arquitetura Cliente-Servidor real**, onde os dados são centralizados e compartilhados em rede;
- Compreender o funcionamento interno e o ciclo de vida de requisições assíncronas utilizando a moderna **Fetch API** nativa com `async/await` e Promises;
- Instalar, configurar e executar microbackends em **Python Flask**, entendendo o papel das rotas HTTP, verbos fundamentais (`GET`, `POST`) e serialização de dados em formato JSON;
- Compreender a diferença arquitetural entre frameworks leves de microsserviços (como o Flask) e pilhas tradicionais completas com servidores de aplicação integrados (como o Apache/XAMPP);
- Identificar e resolver o mecanismo de proteção de segurança **CORS** (*Cross-Origin Resource Sharing*), compreendendo por que o navegador bloqueia chamadas entre origens distintas e como liberar o acesso com `flask-cors`;
- Implementar tratamento defensivo de erros de rede utilizando blocos `try/catch/finally`, garantindo resiliência quando o servidor estiver indisponível;
- Desenvolver interfaces reativas capazes de fornecer feedback visual imediato ao usuário através de **Spinners de Carregamento** e desativação temporária de controles.

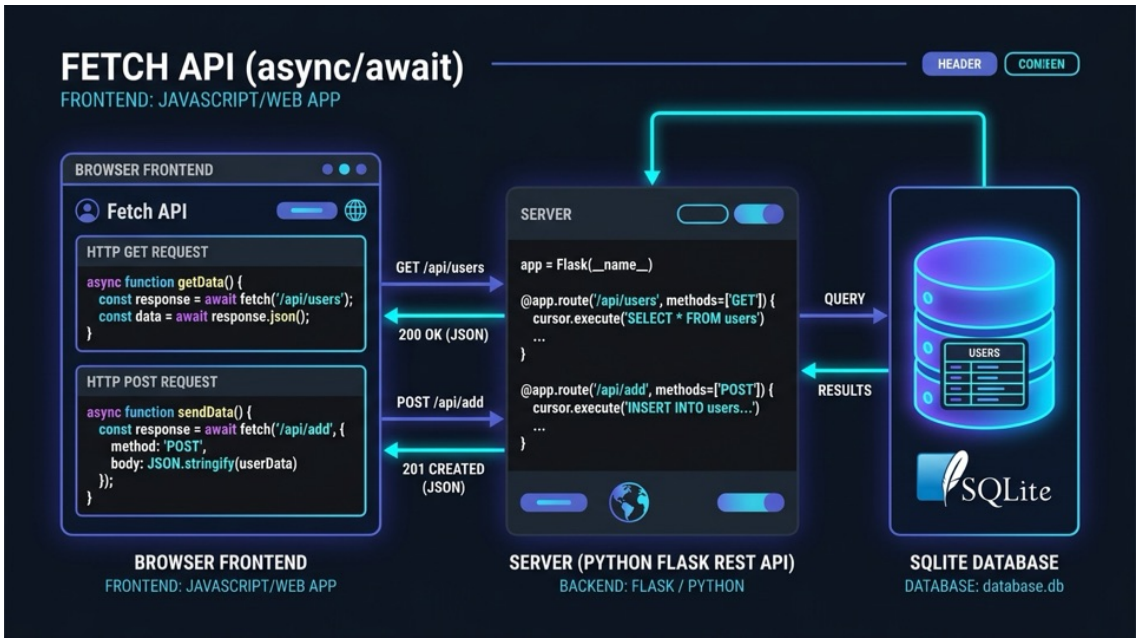


Figura 1: Visão Geral da Aplicação: Vitrine Comunitária conectada via Fetch API com backend RESTful em Flask e persistência relacional SQLite.

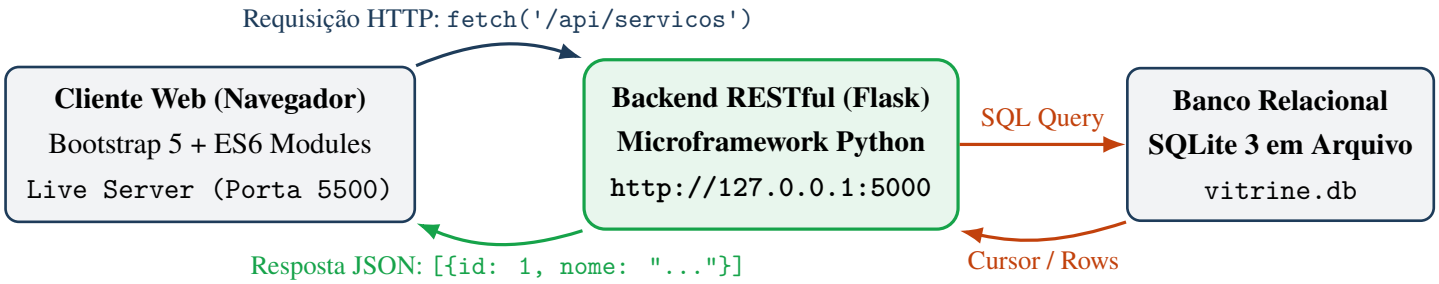
1.1 Introdução: Da Vitrine Isolada à Aplicação em Rede

No **Roteiro Prático 04**, organizamos o frontend na arquitetura modular com ES6 Modules (`services/`, `views/`, `utils/`), porém os dados residiam unicamente no `localStorage` da máquina local. Na prática comercial, os dados precisam ser compartilhados instantaneamente entre diversos computadores e celulares através de um **servidor central com banco de dados**.

Critério de Comparação	**Roteiro 04 (Armazenamento Isolado)**	**Roteiro 05 (Aplicação em Rede)**
Onde os dados residem?	No `localStorage` de cada navegador.	Em banco relacional SQLite (`vitrine.db`) no servidor.
Compartilhamento em rede?	Impossível (dados presos no computador local).	Imediato (qualquer cliente na rede consulta os dados).
Persistência e integridade	Limpeza de histórico ou aba anônima apaga tudo.	Arquivo de banco relacional seguro e persistente no disco.
Arquitetura de software	Monólito no cliente com funções síncronas.	Arquitetura Cliente-Servidor desacoplada com Fetch API e REST.

Tabela 1: Evolução arquitetural do Roteiro 04 para o Roteiro 05.

1.2 Visão Geral da Arquitetura Cliente-Servidor



Camada	Tecnologia	Porta / Endereço	Responsabilidade Principal
Frontend	HTML5, CSS3, JS ES6	Porta 5500 (Live Server)	Renderizar a interface, capturar ações e disparar `fetch()`.
Backend	Python Flask + CORS	Porta 5000 (Localhost)	Roteamento HTTP, validação de regras de negócio e entrega de JSON.
Banco	SQLite 3 em Arquivo	Arquivo `vitrine.db`	Persistência relacional em disco, comandos SQL e integridade ACID.

Tabela 2: Papéis e tecnologias em cada camada da arquitetura.

1.3 Estrutura de Pastas do Projeto Integrado

Para manter o desacoplamento saudável entre a camada de backend e a camada de frontend, organizaremos o projeto na pasta `roteiro-5/exercicio-roteiro5/` com a seguinte estrutura:

Árvore de Diretórios do Projeto Integrado (Full-Stack Didático):

```
roteiro-5/exercicio-roteiro5/
|-- backend/
|   |-- requirements.txt   <-- Dependências Python (Flask, flask-cors)
|   |-- app.py             <-- API RESTful com rotas e conexão SQLite
|   `-- vitrine.db         <-- Banco SQLite (gerado automaticamente no 1º run)
`-- frontend/
    |-- index.html         <-- Interface com status de conexão e spinner
    |-- styles.css         <-- Estilos complementares
    `-- js/
        |-- main.js        <-- Orquestrador assíncrono de eventos
        |-- services/
        |   `-- vitrineService.js <-- Consumo Fetch API com fallback
        |-- views/
        |   `-- vitrineView.js    <-- Renderizador de cards, spinner e toasts
        `-- utils/
            `-- formatters.js     <-- Formatações (100% inalterado!)
```

Princípio do Desacoplamento Arquitetural

Observe atentamente que a camada utilitária (`formatters.js`) e grande parte da camada de visão (`vitrineView.js`) **permanecem exatamente as mesmas do Roteiro 04**. Toda a migração de tecnologia ocorre de forma isolada no `vitrineService.js`. Este é o maior benefício do padrão arquitetural que aprendemos!

1.4 Conceitos Fundamentais em Pílulas Práticas de Aquecimento

Antes de iniciarmos a construção da arquitetura completa da Vitrine Comunitária (que integrará banco de dados relacional persistente, camadas modulares em ES6 e contingência offline), realizaremos **4 mini-práticas guiadas de aquecimento**.

Cada pílula introduz um conceito arquitetural isolado e fornece o código **100% completo** para ser executado imediatamente pelos estudantes no laboratório em poucos minutos.

Aspecto	Diagnóstico Técnico	Solução Implementada no Roteiro
Separação de Papéis	Misturar HTML com lógica JS dificulta manutenção e reaproveitamento.	Arquivos estritamente separados: estrutura no `index.html` e lógica em `functions.js`.
Desafio: Race Condition	Se o script rodar no `<head>` antes do HTML carregar, `document.getElementById` retorna `null`.	Gera o clássico erro fatal: `Cannot read properties of null (reading 'addEventListener')`.
Solução HTML (`defer`)	Bloqueio da renderização da página durante o download do script.	`<script src="functions.js" defer></script>`: download assíncrono em segundo plano e execução após o HTML pronto.
Solução JS (DOM Ready)	Mapeamento de nós em momentos imprevisíveis do ciclo de vida.	`document.addEventListener('DOMContentLoa () => { ... })`: amarra eventos apenas com os nós criados na memória.

Tabela 3: Ciclo de vida do DOM e estratégia de carregamento seguro de scripts.

1.4.1 Pílula 1: Assincronismo e Leitura de Dados com Fetch API (GET)

Fundamentação Teórica: Operações de rede não podem ser executadas de forma síncrona (*bloqueante*), pois o navegador congelaria a tela. O JavaScript moderno utiliza **Promises** com as palavras-chave `async` e `await`. A função `fetch(url)` dispara a requisição e `await resposta.json()` converte o fluxo HTTP em objetos utilizáveis.

Mini-Exemplo 1: Leitura Assíncrona de Arquivo JSON com Fetch GET

Objetivo Prático: Ler um vetor de serviços em formato JSON e preencher dinamicamente uma lista HTML na tela sem recarregar a página, com script externo e controle do DOM.

Código-Fonte Completo no GitHub (Mini-Exemplo 1)

Para visualizar o código cru ou clonar os arquivos deste exemplo diretamente, utilize os atalhos:

- `dados.json` → Abrir dados.json no GitHub
- `index.html` → Abrir index.html no GitHub
- `functions.js` → Abrir functions.js no GitHub

Passo 1.1: Criar o arquivo de dados (`mini-exemplos/01-fetch-get/dados.json`):

```
[
  { "id": 1, "nome": "Maré Alta Artesanatos", "preco": 35.00 },
  { "id": 2, "nome": "Garopaba Web Studio", "preco": 150.00 },
  { "id": 3, "nome": "Pescado Fresco do Zequinha", "preco": 42.00 }
]
```

Passo 1.2: Criar a página HTML estrutural (`mini-exemplos/01-fetch-get/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 1: Fetch API GET</title>
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 1: Fetch API (GET)</h5>
        <p class="text-muted small">Carregamento assíncrono com async/await.</p>

        <button id="btnCarregar" class="btn btn-primary w-100 mb-3">
          Carregar Serviços Locais
        </button>

        <ul id="listaServicos" class="list-group">
          <!-- Itens preenchidos via JavaScript -->
        </ul>
      </div>
    </div>
  </div>
</body>
</html>
```

Passo 1.3: Criar a lógica assíncrona desacoplada (`mini-exemplos/01-fetch-get/functions.js`):

```
// 1. Função assíncrona dedicada
async function carregarServicos(listaServicos) {
  try {
    const resposta = await fetch('dados.json');
    const servicos = await resposta.json();

    listaServicos.innerHTML = '';

    for (const servico of servicos) {
      const item = document.createElement('li');
      item.className =
        'list-group-item d-flex justify-content-between align-items-center';
      item.innerHTML =
        '<span>' + servico.nome + '</span>' +
        '<span class="badge bg-success rounded-pill">R$ ' +
        servico.preco.toFixed(2) + '</span>';
      listaServicos.appendChild(item);
    }

  } catch (erro) {
    console.error('Falha ao carregar os dados:', erro);
  }
}

// 2. Controle seguro do ciclo de vida do DOM
document.addEventListener('DOMContentLoaded', () => {
  const btnCarregar = document.getElementById('btnCarregar');
  const listaServicos = document.getElementById('listaServicos');

  btnCarregar.addEventListener('click', () => {
    carregarServicos(listaServicos);
  });
});
```

Como testar a Pílula 1

Abra o arquivo `index.html` com a extensão **Live Server** do VS Code. Clique no botão **"Carregar Serviços Locais"** e observe os 3 itens serem renderizados instantaneamente no DOM. Abra o Console do DevTools (F12) e confirme que nenhum erro de nó nulo ocorre.

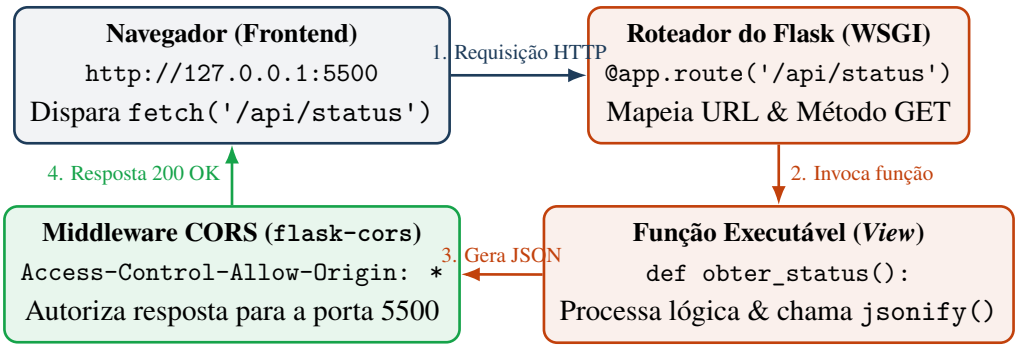
1.4.2 Pílula 2: Microbackend Flask e a Política de Mesma Origem (CORS)

Fundamentação Teórica: Os Três Pilares do Microframework Flask

Em uma arquitetura web moderna desacoplada, os dados são entregues por uma API RESTful remota. Na linguagem Python, o **Flask** é a ferramenta de referência devido a três características essenciais sintetizadas no quadro a seguir:

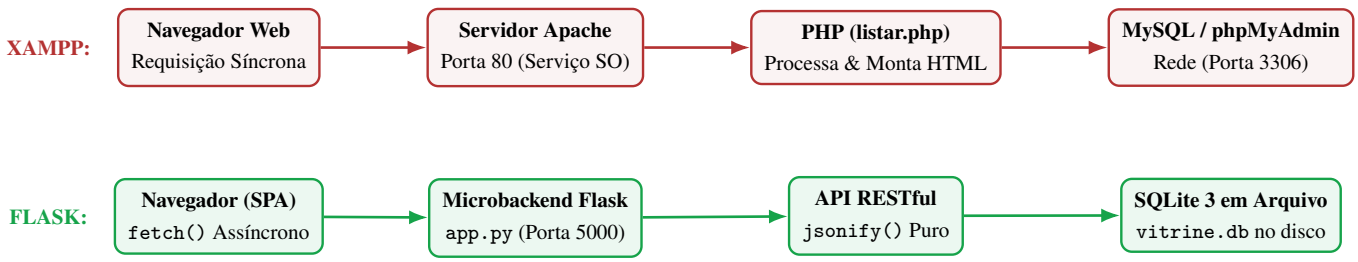
Pilar Conceitual	Características e Funcionamento	Vantagem Didática e Prática
1. Filosofia "Micro"	Núcleo enxuto (*lean core*) baseado no roteador *Werkzeug* e no motor de templates *Jinja2*. Não impõe estruturas rígidas como o Django.	Liberdade total para estruturar o código e focar exclusivamente no roteamento e no JSON.
2. Aplicações Típicas	APIs RESTful para Single Page Applications (SPAs), microsserviços distribuídos em nuvem e APIs preditivas para modelos de IA.	É o padrão consagrado da indústria para criar serviços rápidos, robustos e leves.
3. Ciclo de Vida WSGI	Roteamento declarativo com decoradores Python: `@app.route('/api/...', methods=['GET', 'POST'])` associados a funções executáveis.	Respostas padronizadas em JSON puro via `jsonify()`, com controle explícito de status HTTP.

Tabela 4: Visão sintetizada dos três pilares do microframework Flask.



Comparativo Didático: Pilha Tradicional (XAMPP) vs. Microframework Moderno (Flask)

Muitos estudantes já utilizaram o pacote **XAMPP** (*Apache, MySQL, PHP*) em disciplinas introdutórias. O diagrama e a tabela a seguir mostram o contraste didático entre a arquitetura monolítica clássica e o modelo moderno desacoplado:



Aspecto	Pilha Tradicional (XAMPP / PHP)	Microframework Moderno (Flask / Python)
Servidor Web	Apache como serviço externo do SO (painel XAMPP, portas 80/443).	Servidor HTTP embutido no script Python (`app.run(port=5000)`).
Roteamento	Acoplado à árvore física de arquivos no disco (`/script.php`).	Declarativo, semântico e desacoplado (`@app.route('/api/...')`).
Formato de Saída	Renderização no servidor misturando lógica e HTML (*SSR Monolítico*).	API REST desacoplada retornando dados em JSON puro (`jsonify()`).
Banco de Dados	MySQL em rede gerenciado via navegador no phpMyAdmin.	SQLite embutido em arquivo único (`vitrine.db`), zero configuração externa.
Experiência do Usuário	Recarregamento síncrono da página inteira no navegador.	Requisições assíncronas em segundo plano via Fetch API (`async/await`).

Tabela 5: Quadro comparativo didático entre XAMPP e Flask.

Como verificar e instalar o Flask no computador:

Ação no Terminal	Comandos Executáveis (Windows, macOS e Linux)
1. Verificar Versão	`python -m flask --version` (Windows) / `python3 -m flask --version` (macOS/Linux)
2. Instalar Pacotes	`pip install flask flask-cors` / `python3 -m pip install flask flask-cors`
3. Por que o CORS?	O navegador bloqueia requisições entre origens com portas distintas (`5500` vs `5000`) pela Política de Mesma Origem (SOP). O `flask-cors` anexa o cabeçalho liberador: `Access-Control-Allow-Origin: *`.

Tabela 6: Guia prático de comandos de terminal para configuração do Flask.

Mini-Exemplo 2: Microservidor Flask & Consumo Cross-Origin

Objetivo Prático: Executar um microservidor Python em porta distinta e consumi-lo via Fetch a partir do navegador, compreendendo a atuação do CORS com script desacoplado.

Código-Fonte Completo no GitHub (Mini-Exemplo 2)

Para visualizar o código cru ou clonar os arquivos deste exemplo diretamente, utilize os atalhos:

- `server.py` → Abrir server.py no GitHub
- `index.html` → Abrir index.html no GitHub
- `functions.js` → Abrir functions.js no GitHub

Passo 2.1: Criar o microservidor Python (`mini-exemplos/02-micro-flask-cors/server.py`):

```python
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
# Libera o acesso entre portas distintas (5500 e 5000)
CORS(app)

@app.route('/api/status', methods=['GET'])
def obter_status():
    return jsonify({
        "status": "online",
        "servidor": "Micro API Flask",
        "cidade": "Garopaba - SC"
    })

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Passo 2.2: Criar a página consumidora (`mini-exemplos/02-micro-flask-cors/index.html`):

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 2: Flask API & CORS</title>
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 2: Flask API & CORS</h5>
        <p class="text-muted small">
          Consumindo microbackend Flask (porta 5000) via Live Server (porta 5500).
        </p>
```

```
        <button id="btnConectar" class="btn btn-dark w-100 mb-3">
          Verificar Status da API
        </button>

        <div id="resultadoStatus" class="alert alert-secondary mb-0">
          Aguardando conexão com a API...
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

Passo 2.3: Criar a lógica de consumo assíncrono (`mini-exemplos/02-micro-flask-cors/functions.`

```
// 1. Função assíncrona nomeada que consulta a API Flask
async function consultarStatusApi(resultadoStatus) {
  try {
    const resposta = await fetch('http://127.0.0.1:5000/api/status');
    const dados = await resposta.json();

    resultadoStatus.className = 'alert alert-success mb-0';
    resultadoStatus.innerHTML =
      '<strong>Status:</strong> ' + dados.status + '<br>' +
      '<strong>Servidor:</strong> ' + dados.servidor + '<br>' +
      '<strong>Origem:</strong> ' + dados.cidade;

  } catch (erro) {
    resultadoStatus.className = 'alert alert-danger mb-0';
    resultadoStatus.innerHTML =
      '<strong>Erro de Conexão:</strong> Não foi possível acessar a API.<br>' +
      '<small>Verifique se o server.py está rodando e se o CORS está liberado.</small>';
  }
}

// 2. Controle seguro de carregamento do DOM
document.addEventListener('DOMContentLoaded', () => {
  const btnConectar = document.getElementById('btnConectar');
  const resultadoStatus = document.getElementById('resultadoStatus');

  btnConectar.addEventListener('click', () => {
    consultarStatusApi(resultadoStatus);
  });
});
```

Como testar a Pílula 2 e observar o CORS em ação

1. No terminal do VS Code, execute: `python server.py`.
2. Abra o `index.html` no Live Server e clique em **"Verificar Status da API"**. O card exibirá a confirmação verde da cidade de Garopaba.
3. **Desafio Didático em Aula:** No `server.py`, comente temporariamente a linha `# CORS(app)` e reinicie o servidor. Ao clicar no botão, abra a aba *Console* do DevTools (F12) e observe o

bloqueio em vermelho por violação de CORS! Em seguida, descomente a linha.

1.4.3 Pílula 3: Envio de Dados com Fetch POST e JSON

Fundamentação Teórica: Os Três Mandamentos do Envio de Dados com POST

Enquanto requisições **GET** servem para recuperar dados, o método HTTP **POST** transfere novas informações do cliente para persistência no servidor. Para enviar objetos estruturados com a Fetch API, três etapas são mandatórias:

Etapa Mandatória	Sintaxe de Código	Papel no Protocolo HTTP
1. Método Semântico	`method: 'POST'`	Informa ao servidor que se trata de uma operação de criação/cadastro.
2. Cabeçalho de Tipo	`headers: { 'Content-Type': 'application/json' }`	Avisa ao Flask que o corpo trafega codificado no padrão MIME JSON.
3. Serialização	`body: JSON.stringify(dadosParaEnvio)`	Converte o objeto JavaScript em memória em texto puro JSON para a rede.

Tabela 7: Os três mandamentos para envio de dados via Fetch API POST.

Mini-Exemplo 3: Cadastro de Serviços com Fetch POST

Objetivo Prático: Submeter um formulário HTML e enviar os dados para gravação na API Flask via requisição POST, com validação e script desacoplado.

Código-Fonte Completo no GitHub (Mini-Exemplo 3)

Para visualizar o código cru ou clonar os arquivos deste exemplo diretamente, utilize os atalhos:

- `server.py` → Abrir server.py no GitHub
- `index.html` → Abrir index.html no GitHub
- `functions.js` → Abrir functions.js no GitHub

Passo 3.1: Criar o servidor com rota POST (`mini-exemplos/03-fetch-post/server.py`):

```
from flask import Flask, jsonify, request
```

```
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

servicos_cadastrados = []

@app.route('/api/cadastrar', methods=['POST'])
def cadastrar_servico():
    dados = request.get_json()

    if not dados or 'nome' not in dados or 'preco' not in dados:
        return jsonify({"erro": "Campos obrigatórios ausentes."}), 400

    novo_id = len(servicos_cadastrados) + 1
    registro = {
        "id": novo_id,
        "nome": str(dados['nome']).strip(),
        "preco": float(dados['preco'])
    }
    servicos_cadastrados.append(registro)
    print(f"[API FLASK] Novo serviço cadastrado: {registro}")

    return jsonify({
        "mensagem": "Serviço cadastrado com sucesso no servidor!",
        "registro": registro,
        "total": len(servicos_cadastrados)
    }), 201

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Passo 3.2: Criar o formulário frontend (`mini-exemplos/03-fetch-post/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 3: Fetch API POST</title>
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 3: Fetch API (POST)</h5>
        <p class="text-muted small">
          Enviando dados estruturados em JSON via requisição POST.
        </p>

        <form id="formCadastro">
          <div class="mb-3">
            <label for="campoNome" class="form-label">Nome do Serviço:</label>
            <input
              type="text"
```

```
            id="campoNome"
            class="form-control"
            placeholder="Ex: Conserto de Pranchas"
            required>
        </div>

        <div class="mb-3">
          <label for="campoPreco" class="form-label">Preço Base (R$):</label>
          <input
            type="number"
            id="campoPreco"
            class="form-control"
            step="0.01"
            placeholder="Ex: 80.00"
            required>
        </div>

        <button type="submit" class="btn btn-success w-100 mb-3">
          Enviar via POST para a API
        </button>
      </form>

      <div id="feedbackPost" class="alert alert-secondary d-none mb-0"></div>
    </div>
  </div>
 </div>
</body>
</html>
```

Passo 3.3: Criar a lógica de envio assíncrono (`mini-exemplos/03-fetch-post/functions.js`):

```
// 1. Função assíncrona nomeada para processar o envio
async function cadastrarNovoServico(
  evento,
  formCadastro,
  campoNome,
  campoPreco,
  feedbackPost
) {
  evento.preventDefault();

  const dadosParaEnvio = {
    nome: campoNome.value.trim(),
    preco: parseFloat(campoPreco.value)
  };

  try {
    const resposta = await fetch(
      'http://127.0.0.1:5000/api/cadastrar',
      {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify(dadosParaEnvio)
      }
    );

    // Validação explícita de status
    if (resposta.ok === false) {
      throw new Error('Erro no servidor: ' + resposta.status);
    }

    const dadosResposta = await resposta.json();

    feedbackPost.className = 'alert alert-success mb-0';
    feedbackPost.classList.remove('d-none');
    feedbackPost.innerHTML =
      '<strong>' + dadosResposta.mensagem + '</strong><br>' +
      '<small>ID gerado: #' + dadosResposta.registro.id +
      ' | Total: ' + dadosResposta.total + '</small>';

    formCadastro.reset();

  } catch (erro) {
    feedbackPost.className = 'alert alert-danger mb-0';
    feedbackPost.classList.remove('d-none');
    feedbackPost.innerHTML =
      '<strong>Falha ao enviar:</strong> ' + erro.message;
  }
}

// 2. Controle seguro de carregamento do DOM
document.addEventListener('DOMContentLoaded', () => {
  const formCadastro = document.getElementById('formCadastro');
  const campoNome = document.getElementById('campoNome');
  const campoPreco = document.getElementById('campoPreco');
  const feedbackPost = document.getElementById('feedbackPost');

  formCadastro.addEventListener('submit', (evento) => {
```

```
    cadastrarNovoServico(
      evento,
      formCadastro,
      campoNome,
      campoPreco,
      feedbackPost
    );
  });
});
```

Como testar a Pílula 3

Com o `server.py` em execução, digite um serviço e preço no formulário e clique em **"Enviar via POST"**. Observe que o terminal do Python exibirá o log do registro recebido e a página exibirá o alerta verde com o ID gerado pelo servidor!

1.4.4 Pílula 4: Resiliência de Rede e Feedback Visual (try/catch/finally com Spinner)

Fundamentação Teórica: Anatomia da Resiliência de Rede

Falhas de conectividade (servidor offline, queda de rede, lentidão) são comuns no mundo real. Uma aplicação de qualidade profissional nunca deixa a interface congelada sem resposta.

Bloco / Recurso	Momento de Disparo	Papel na Experiência do Usuário (UX)
1. Bloco `try`	Tentativa inicial de requisição	Dispara o `fetch()` e processa os dados com `await`.
2. Validação `ok`	Imediatamente após a resposta	Verifica se o código HTTP é da faixa 200–299; se for falso, lança erro.
3. Bloco `catch`	Quando ocorre falha de rede/servidor	Intercepta a exceção sem quebrar o app e exibe alerta amigável.
4. Bloco `finally`	**Executado SEMPRE**	Oculta o spinner e reativa os botões para nova tentativa do usuário.

Tabela 8: Ciclo de resiliência e tratamento defensivo com try...catch...finally.

Mini-Exemplo 4: Controle de Loading State e Tratamento de Quedas de Rede

Objetivo Prático: Implementar um botão com spinner de carregamento e tratar quedas de conectividade com feedback amigável ao usuário.

Código-Fonte Completo no GitHub (Mini-Exemplo 4)

Para visualizar o código cru ou clonar os arquivos deste exemplo diretamente, utilize os atalhos:

- `index.html` → Abrir index.html no GitHub
- `functions.js` → Abrir functions.js no GitHub

Passo 4.1: Criar a interface resiliente (`mini-exemplos/04-fetch-spinner/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 4: Tratamento de Falhas & Spinners</title>
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css"
    rel="stylesheet">
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 520px;">
    <div class="card shadow-sm">
```

```
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 4: Resiliência & Spinner</h5>
        <p class="text-muted small">
          Controle de loading states e tratamento com try...catch...finally.
        </p>

        <button
          id="btnVerificar"
          class="btn btn-primary w-100 d-flex justify-content-center align-items-center gap-2 mb-3">
          <span id="iconeBotao"><i class="bi bi-arrow-repeat"></i></span>
          <span
            id="spinnerCarregamento"
            class="spinner-border spinner-border-sm d-none"
            role="status"></span>
          <span id="textoBotao">Testar Conexão com o Servidor</span>
        </button>

        <div id="caixaAlerta" class="alert d-none mb-0" role="alert"></div>
      </div>
    </div>
  </div>
</body>
</html>
```

Passo 4.2: Criar as funções de estado e resiliência (`mini-exemplos/04-fetch-spinner/functions.js`)

```
// 1. Funções auxiliares para clareza visual de estados
function ativarEstadoCarregando(elementos) {
  elementos.btnVerificar.disabled = true;
  elementos.iconeBotao.classList.add('d-none');
  elementos.spinnerCarregamento.classList.remove('d-none');
  elementos.textoBotao.textContent = 'Consultando Servidor...';
  elementos.caixaAlerta.classList.add('d-none');
}

function desativarEstadoCarregando(elementos) {
  elementos.spinnerCarregamento.classList.add('d-none');
  elementos.iconeBotao.classList.remove('d-none');
  elementos.textoBotao.textContent = 'Testar Conexão com o Servidor';
  elementos.btnVerificar.disabled = false;
  elementos.caixaAlerta.classList.remove('d-none');
}

// 2. Função assíncrona nomeada com bloco defensivo try...catch...finally
async function testarConexaoComServidor(elementos) {
  ativarEstadoCarregando(elementos);

  try {
    const resposta = await fetch('http://127.0.0.1:5000/api/status');

    if (resposta.ok === false) {
      throw new Error('Código de erro HTTP: ' + resposta.status);
    }

    const dados = await resposta.json();

    elementos.caixaAlerta.className = 'alert alert-success mb-0';
    elementos.caixaAlerta.innerHTML =
      '<i class="bi bi-check-circle-fill me-1"></i> ' +
```

```
      '<strong>API Online!</strong> Conectado ao servidor (' +
      dados.servidor + ').';

  } catch (erro) {
    elementos.caixaAlerta.className = 'alert alert-danger mb-0';
    elementos.caixaAlerta.innerHTML =
      '<i class="bi bi-exclamation-triangle-fill me-1"></i> ' +
      '<strong>Falha de Conectividade:</strong> Não foi possível acessar a API.<br>' +
      '<small class="text-muted">Detalhe: ' + erro.message + '</small>';

  } finally {
    // Executado SEMPRE: desliga o spinner e reativa o botão
    desativarEstadoCarregando(elementos);
  }
}

// 3. Controle seguro de carregamento do DOM
document.addEventListener('DOMContentLoaded', () => {
  const elementos = {
    btnVerificar: document.getElementById('btnVerificar'),
    iconeBotao: document.getElementById('iconeBotao'),
    spinnerCarregamento: document.getElementById('spinnerCarregamento'),
    textoBotao: document.getElementById('textoBotao'),
    caixaAlerta: document.getElementById('caixaAlerta')
  };

  elementos.btnVerificar.addEventListener('click', () => {
    testarConexaoComServidor(elementos);
  });
});
```

Como testar a Resiliência da Pílula 4

1. **Cenário de Sucesso:** Com o `server.py` da Pílula 2 rodando na porta 5000, clique no botão. Observe o spinner girar rapidamente e o alerta verde confirmar o status.

2. **Cenário de Queda de Rede (Servidor Desligado):** Pressione `Ctrl+C` no terminal para derrubar o `server.py`. Clique no botão novamente. Observe que o spinner gira e, graças ao bloco `catch`, a tela **não congela**: um alerta vermelho informa a falha de conectividade e o bloco `finally` restaura o botão para novo teste!

Conclusão da Parte 1 e Continuação na Tarefa Guiada (Parte 2)

Parabéns! Você completou com sucesso todos os fundamentos conceituais e os 4 mini-exemplos de aquecimento do Roteiro 05:

- **Pílula 1:** Fetch API consumindo endpoints públicos assíncronos com JavaScript moderno;
- **Pílula 2:** Microbackend em Python Flask com controle explícito de cabeçalhos CORS;
- **Pílula 3:** Submissão de dados via `POST` com corpo JSON (`application/json`);
- **Pílula 4:** Feedback de carregamento com spinners e tratamento defensivo com

`try/catch/finally`.

Próximo Passo: Abra o arquivo **Roteiro Prático 05 (Parte 2) — Tarefa Guiada** (`roteiro-5/tarefa-guiada.pdf`), no qual você aplicará todos esses recursos para refatorar o projeto completo da **Vitrine Comercial de Serviços**, integrando persistência em banco relacional SQLite e modo de contingência offline!